

Project - Low Level Design

on

IaC Provisioning for Finance System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	KRATI PATIDAR	EN22CS301514
2	KAMAL YADAV	EN22CS301476
3	KASHISH SHARMA	EN22CS301492
4	KHUSHI MEHRA	EN22CS301504
5	JIDNYAS BHUSHAN BHONGE	EN22CS301469

Group Name: Group 03D5

Project Number: DO-03

Industry Mentor Name: Mr.Vaibhav

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 2025–2026

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. References

1. Introduction

1.1 Scope of the Document

This document provides a detailed low-level design for automating the deployment of a Finance Support System using DevOps practices. It includes infrastructure provisioning, containerization, and continuous deployment.

1.2 Intended Audience

- DevOps Engineers
- Developers
- System Administrators
- Evaluators

1.3 System Overview

The system automates the deployment of a Finance Support Triage Application using:

- Docker for containerization
- AWS EC2 for cloud hosting
- GitHub for version control
- GitHub Actions for CI/CD
- Terraform for Infrastructure as Code (IaC)

The goal is to eliminate manual setup and ensure consistent, repeatable deployments.

2. System Design

2.1 Application Design

Architecture Type:

- Containerized Application Architecture

Layers:

1. Source Code Layer

- Hosted on GitHub
- Contains application code + Dockerfile

2. Build Layer

- Docker builds image from source

3. Deployment Layer

- AWS EC2 instance runs Docker container

4. Automation Layer

- CI/CD pipeline automates deployment

2.2 Process Flow

1. Developer pushes code to GitHub
2. GitHub Actions triggers CI/CD pipeline
3. Docker image is built automatically
4. Infrastructure is provisioned using Terraform
5. EC2 instance is created/configured
6. Docker container is deployed on EC2
7. Application becomes accessible via public IP

2.3 Information Flow

Developer → GitHub → CI/CD Pipeline → Docker Image → AWS EC2 → User Browser

- Input: Source code updates
- Processing: Build + deploy automation
- Output: Live application

2.4 Components Design

1. GitHub (Version Control)

- Stores codebase
- Maintains Dockerfile and configs

2. Docker

- Builds application image
- Ensures consistent runtime environment

3. AWS EC2

- Virtual machine hosting the application
- Runs Docker container

4. Terraform (IaC)

- Automates infrastructure creation
- Defines EC2 instance in code

5. CI/CD (GitHub Actions)

- Automates:
 - Build process
 - Deployment process

6. Application (Finance System)

- Streamlit-based UI
- AI-driven email triage system

2.5 Key Design Considerations

- Automation: Eliminates manual setup
- Scalability: Can extend to multiple instances
- Portability: Docker ensures environment consistency
- Reliability: Automated deployment reduces errors
- Security: SSH key-based authentication for EC2
- Maintainability: Code-driven infrastructure

2.6 API Catalogue

API	Method	Description
/	GET	Loads application dashboard
/email-fetch	IMAP	Fetch emails
/analyze	GET	AI-based classification
/send-reply	SMTP	Sends response email

3. References

1. AWS EC2 Documentation
2. Docker Documentation
3. Terraform Documentation
4. GitHub Actions Documentation
5. Streamlit Documentation

Conclusion

This project demonstrates the implementation of DevOps practices for automating infrastructure provisioning and application deployment. It ensures a scalable, reliable, and reproducible system using modern tools like Docker, AWS, and Terraform.